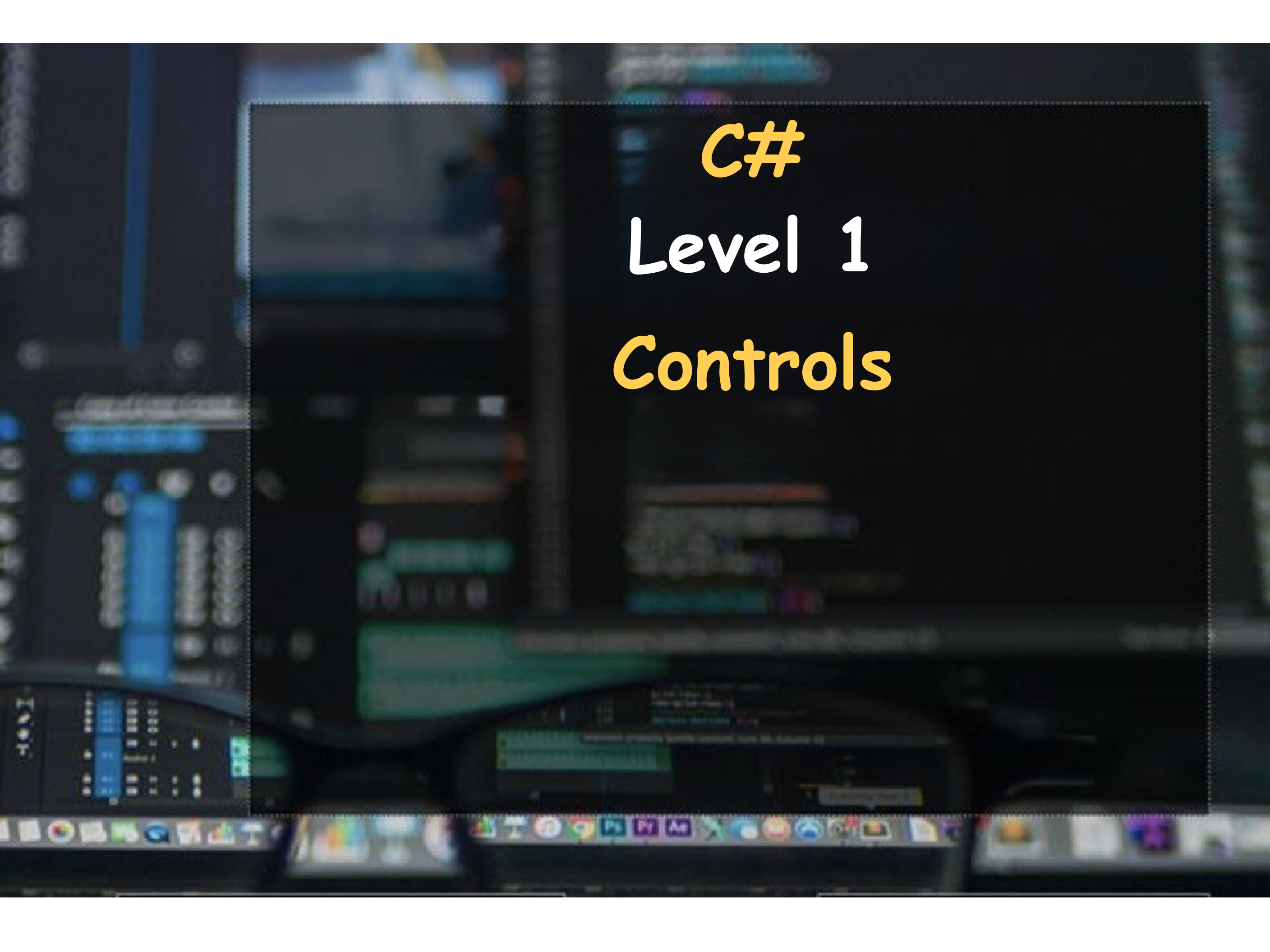




C#

Level 1

Controls



C#

Level 1

Controls

Definition

Control

 A Control is a visual component placed on a Form to interact with the user.

Common Controls:

- ◆ TextBox → enters or displays text.
- ◆ Button → performs an action.
- ◆ Label → displays text.
- ◆ CheckBox → selects/unselects an option.
- ◆ RadioButton → selects an option from related choices.
- ◆ ComboBox → provides a drop-down list.
- ◆ GroupBox → organizes related controls together.

WinForms Designer

Toolbox

 Contains Controls that can be added to the Form.

Form Designer

 Visual area used to design the application.

Properties Window

 Used to view and change Control properties.

Solution Explorer

 Shows the project files.

■ Important Control Properties

■ Name

- Identifies the Control in C# code.

Example:

```
Name = txtName  
txtName.Text
```

■ Text

- Gets or changes the text displayed by the Control.
`textBox1.Text = "Hello";`

■ Location

- Determines the Control's position on the Form.
`button1.Location = new Point(200, 100);`
- X = horizontal position.
- Y = vertical position.

■ Size

- Determines the Control's width and height.
`button1.Size = new Size(200, 50);`
- 200 = Width.
- 50 = Height.

■ Enabled

- Determines whether the user can interact with the Control.
`textBox1.Enabled = false;`
- false → Control remains visible but cannot be interacted with.

■ Important Control Properties

■ Visible

■ Determines whether the Control is displayed.

`textBox1.Visible = false;`

■ `false` → Control is hidden.

■ BackColor

■ Changes the background color.

`textBox1.BackColor = Color.Red;`

■ ForeColor

■ Changes the text/foreground color.

`textBox1.ForeColor = Color.Red;`

■ Font

■ Controls font family, size, and style.

`label1.Font = new Font("Arial", 20, FontStyle.Bold);`

■ TextAlign

■ Controls where the text appears inside the Control.

For TextBox:

`textBox1.TextAlign = HorizontalAlignment.Center;`

■ Common values:

Left, Center, Right

For Button and Label:

`button1.TextAlign = ContentAlignment.MiddleCenter;`

■ Important Control Properties

■ ReadOnly

■ Prevents the user from modifying the TextBox content while still allowing interaction such as selecting/copying text.

`textBox1.ReadOnly = true;`

■ Multiline

■ Allows a TextBox to contain multiple lines.

`textBox1.Multiline = true;`

■ MaxLength

■ Limits the number of characters entered into a TextBox.

■ PasswordChar

■ Displays entered characters as password characters.

■ TabIndex

■ Determines the order of Controls when navigating with the Tab key.

■ TabStop

■ Determines whether the Control participates in Tab navigation.

TabIndex → "When?"

TabStop → "Can I participate?"

■ Tag

■ Stores additional information associated with a Control.

Font Formatting

FontStyle

 Controls the style of the text.

Common styles:

Regular

Bold

Italic

Underline

Strikeout

Example:

```
label1.Font = new Font("Arial", 20, FontStyle.Bold);
```

 Multiple styles can be combined using |.

```
label1.Font = new Font(  
    "Arial",  
    20,  
    FontStyle.Bold | FontStyle.Italic  
);
```

TextBox

 Used to enter or display text.

Important properties:

Text

Name

Location

Size

Enabled

Visible

BackColor

ForeColor

Font

TextAlign

ReadOnly

Multiline

MaxLength

PasswordChar

Note

- ◆ `ReadOnly = true` → user can see/select text but cannot edit it.
- ◆ `Enabled = false` → user cannot interact with the Control.
- ◆ `Visible = false` → Control is hidden.

CheckBox

Definition

◆ Allows the user to select or unselect an independent option.

Checked

 Returns true if selected and false if unselected.

`chkReceive.Checked`

Example:

```
MessageBox.Show(chkReceive.Checked.ToString());
```

 Checked is a Boolean value.

 ToString() converts the Boolean value into text.

RadioButton

Definition

◆ Allows the user to select an option from related choices.

Checked

 Returns true if the RadioButton is selected.
`rdBtnSmall.Checked`

Note

- ◆ RadioButtons are commonly used together when the user should choose between related options.
- ◆ A GroupBox can be used to organize related RadioButtons.

GroupBox

Definition

- ◆ A container used to organize related Controls together.
- ◆ It is especially useful for grouping related RadioButtons.

Events

◆ An Event is something that happens to a Control and can trigger C# code.

Click

 Occurs when the user clicks a Control.

```
private void button1_Click(object sender, EventArgs e)
{
    textBox2.Text = textBox1.Text;
}
```

MouseEnter

 Occurs when the mouse pointer enters a Control.

TextChanged

 Occurs when the text inside a TextBox changes.

```
private void textBox1_TextChanged(object sender, EventArgs e)
{
    textBox2.Text = textBox1.Text;
}
```

💡 Other examples:

MouseLeave

DoubleClick

KeyDown

KeyPress

Properties vs Events

Property

- ✓ Describes or controls the appearance, state, or behavior of a Control.
- ✓ Examples: Text, Enabled, Visible, Font, BackColor.

Event

- ✓ Represents something that happens to a Control.
- ✓ Examples: Click, MouseEnter, TextChanged.

☰ Event Handler

🔍 Definition

- ◆ An Event Handler is the method that executes when an Event occurs.

Example:

```
private void button1_Click(object sender, EventArgs e)
{
    textBox2.Text = textBox1.Text;
}
```

- ◆ Visual Studio can create Event Handlers through the Properties Window → Events ⚡.

WinForms Project Files

Form1.cs

 Normally contains application logic and Event Handlers.

Form1.Designer.cs

 Contains code generated by the WinForms Designer.

 Usually contains Control creation and visual properties.

Program.cs

 Contains the application entry point and starts the main Form.

Example:

```
Application.Run(new Form1());
```

Note

◆ Visual changes should normally be made through the Designer and Properties Window instead of manually editing Form1.Designer.cs.

Creating Another Form

◆ A WinForms application can contain multiple Forms.

Procedure

1. Right-click the project in Solution Explorer.
2. Select Add → Windows Form.
3. Give the Form a name.
4. Click Add.

Creating a Form Object

```
Form frm1 = new Form1();
```

 Form1 → the Form class.

 frm1 → an object created from Form1.

Show()

Definition

- ◆ Opens another Form as a modeless window.

Form frm1 = new Form1();

frm1.Show();

- ▨ The main Form can still be used while the new Form is open.

Show() → Open and continue

ShowDialog()

Definition

- ◆ Opens another Form as a modal dialog.

Form frm1 = new Form1();

frm1.ShowDialog();

- ◆ The user normally cannot interact with the main Form until the dialog is closed.

ShowDialog() → Open and wait

Close()

Definition

- ◆ Closes the current Form.
 `this.Close();`
- ◆ `this` refers to the current Form/object.

MessageBox

Definition

◆ MessageBox is a small dialog used to display information or ask the user for a simple response.

```
MessageBox.Show("Hello");
```

Message

 Text displayed to the user.

Title

 Text displayed in the MessageBox title bar.

```
MessageBox.Show("Hello", "Title");
```

Buttons

 Determines which buttons are displayed.

```
MessageBoxButtons.YesNo
```

Icon

 Displays an icon in the MessageBox.

```
MessageBoxIcon.Exclamation
```

Default Button

 Determines which button is selected by default.

```
MessageBoxDefaultButton.Button2
```

Checking MessageBox Result

- ◆ MessageBox can return a DialogResult indicating the user's choice.

Example:

```
if (MessageBox.Show(
    "Do you want to continue?",
    "Confirm",
    MessageBoxButtons.YesNo) == DialogResult.Yes)
{
    MessageBox.Show("You clicked Yes.");
}
else
{
    MessageBox.Show("You clicked No.");
}
```

Important Form Concepts

this

 Refers to the current object/Form.

Examples:

this.Text

this.BackColor

this.Close();

InitializeComponent()

 Initializes the Form and its Controls.

new Form1()

 Creates a new object of Form1.

✦ Summary

- ✓ Control → visual component placed on a Form.
- ✓ Property → controls or describes a Control.
- ✓ Event → something that happens to a Control.
- ✓ Event Handler → method executed when an Event occurs.
- ✓ Text → gets or changes displayed text.
- ✓ Name → identifies a Control in code.
- ✓ Location → controls position.
- ✓ Size → controls width and height.
- ✓ Enabled → controls user interaction.
- ✓ Visible → controls whether the Control is displayed.
- ✓ BackColor → background color.
- ✓ ForeColor → text color.
- ✓ Font → font family, size, and style.

✦ Summary

- ✓ `TextAlign` → text position inside the Control.
- ✓ `ReadOnly` → prevents editing while keeping interaction available.
- ✓ `Multiline` → allows multiple lines in a `TextBox`.
- ✓ `TabIndex` → controls Tab navigation order.
- ✓ `TabStop` → controls whether the Control participates in Tab navigation.
- ✓ `CheckBox` → independent selection.
- ✓ `RadioButton` → selection between related options.
- ✓ `GroupBox` → organizes related Controls.
- ✓ `Tag` → stores additional information.
- ✓ `Click` / `MouseEnter` / `TextChanged` → common Events.
- ✓ `Show()` → opens a modeless Form.
- ✓ `ShowDialog()` → opens a modal Form.
- ✓ `Close()` → closes the current Form.
- ✓ `MessageBox.Show()` → displays a dialog to the user.

Thank you 😊